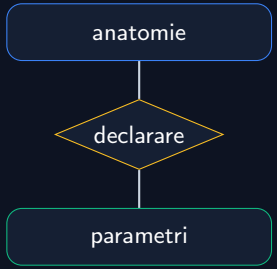
# Funcții în C/C++

## Anatomie, Declarare și Parametri

Învață să structurezi codul folosind subprograme și controlând modul în care datele sunt transmise.

Săptămâna 6



# Anatomia unei funcții C/C++

## STRUCTURĂ DE BAZĂ

O funcție este un subprogram care primește date, efectuează anumite operații și, opțional, returnează un rezultat.

Structura generală a unei funcții cuprinde:

- **Tipul returnat:** tipul de date al rezultatului (sau `void` dacă nu returnează nimic).
- **Numele funcției:** identificatorul folosit pentru apel.
- **Lista de parametri:** variabilele care primesc valorile la apel (poate fi vidă).
- **Corpul funcției:** instrucțiunile dintre `{}`.

```
int suma(int a, int b) {
    int s = a + b;
    return s;
}

void salut() {
    cout << "Buna!";
}
```

# Definirea și declararea funcțiilor

## ANTET ȘI IMPLEMENTARE

Pentru a fi folosită, o funcție trebuie să fie cunoscută de compilator înainte de a fi apelată. Avem două concepte distincte:

### Declararea (Prototipul)

Reprezintă antetul funcției, urmat de punct și virgulă `;`. Anunță compilatorul despre existența funcției, tipul returnat și parametrii ei, fără a da implementarea.

```
int maxim(int x, int y);
```

### Definirea (Implementarea)

Cuprinde antetul funcției și corpul acesteia (între acolade). O funcție nu poate fi definită de mai multe ori, dar poate fi declarată de mai multe ori.

```
int maxim(int x, int y) {...}
```

# Transmiterea parametrilor prin valoare

### PRIN VALOARE (BY VALUE)

La apelul funcției, parametrii formali (din definiție) primesc o **copie** a valorilor parametrilor efectivi (din apel).

**Atenție:**
Orice modificare a parametrilor în interiorul funcției **nu va afecta** variabilele originale din exterior (din programul apelant).

```
void dublare(int x) {
    x = x * 2;
    cout << "In functie: " << x << endl;
}

int main() {
    int a = 5;
    dublare(a); // Afiseaza: 10
    cout << "In main: " << a; //
        Afiseaza: 5
    return 0;
}
```

# Transmiterea parametrilor prin referință

## PRIN REFERINȚĂ (BY REFERENCE)

Folosind operatorul & înaintea numelui parametrului formal, funcția primește acces direct la zona de memorie a variabilei originale.

**Avantaj principal:**

Modificările făcute asupra parametrului formal **se reflectă direct** în variabila originală transmisă la apel.

```
void interschimbare(int &x, int &y) {
    int aux = x;
    x = y;
    y = aux;
}

int main() {
    int a = 3, b = 7;
    interschimbare(a, b);
    // a devine 7, b devine 3
    return 0;
}
```

# Exercițiu recapitulativ

## TESTEAZĂ-ȚI CUNOȘTINȚELE

### 1. Modificarea valorilor

Ce va afișa următorul program? Observă prezența sau absența simbolului `&`.

```
void test(int a, int &b) {
    a++;
    b++;
}
int main() {
    int x = 1, y = 1;
    test(x, y);
    cout << x << " " << y;
}
```

### Explicație

**Răspuns corect:** `1 2`

De ce?

`x` este transmis **prin valoare**, deci modificarea parametrului `a` nu afectează variabila `x`.

`y` este transmis **prin referință** (datorită `&b`), deci modificarea parametrului `b` se reflectă în variabila `y`, care devine 2.